



Lecturer,
Dept. of CSE, ULAB, Dhaka



EMAIL:
atanu.shuvam@ulab.edu.bd

CSE 2201 : Algorithms

Lecture 03: Algorithms Ultra Pro Max

Atanu Shuvam Roy



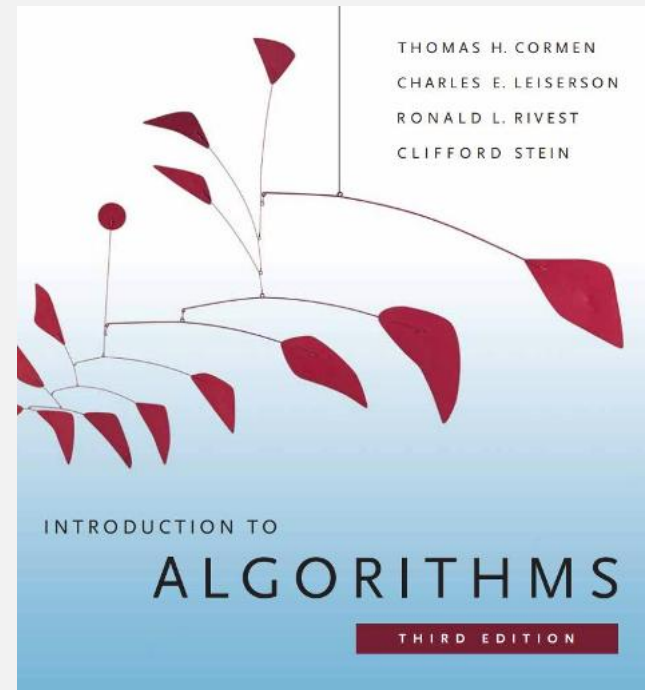
Class Code
GVWTHYJV



Fall 2025

Reference Book

- Introduction to Algorithms, 3rd edition. T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein



Distribution of Marks

Assessments	%
Mid Term Examination	25
Final Examination	40
Attendance & class performance	10
Assignment & Quizzes	25
Total	100



PRINCETON
UNIVERSITY

*“ It is convenient to have a **measure of the amount of work involved in a computing process**, even though it be a very **crude one**. We may count up the number of times that various elementary operations are applied in the whole process and then given them various weights. We might, for instance, count the number of additions, subtractions, multiplications, divisions, recording of numbers, and extractions of figures from tables. In the case of computing with matrices most of the work consists of multiplications and writing down numbers, and **we shall therefore only attempt to count the number of multiplications and recordings.** ” — Alan Turing*

ROUNDING-OFF ERRORS IN MATRIX PROCESSES

By A. M. TURING

(National Physical Laboratory, Teddington, Middlesex)

[Received 4 November 1947]

SUMMARY

A number of methods of solving sets of linear equations and inverting matrices are discussed. The theory of the rounding-off errors involved is investigated for some of the methods. In all cases examined, including the well-known 'Gauss elimination process', it is found that the errors are normally quite moderate: no exponential build-up need occur.



Simplification 1: cost model

Cost model. Use some basic operation as a proxy for running time.

```
int count = 0;
for (int i = 0; i < N; i++)
    for (int j = i+1; j < N; j++)
        if (a[i] + a[j] == 0)
            count++;
```

$$0 + 1 + 2 + \dots + (N - 1) = \frac{1}{2} N (N - 1) \\ = \binom{N}{2}$$

operation	frequency
variable declaration	$N + 2$
assignment statement	$N + 2$
less than compare	$\frac{1}{2} (N + 1) (N + 2)$
equal to compare	$\frac{1}{2} N (N - 1)$
array access	$N (N - 1)$
increment	$\frac{1}{2} N (N - 1)$ to $N (N - 1)$

cost model = array accesses
(we assume compiler/JVM do not optimize array accesses away!)

Simplification 2: tilde notation

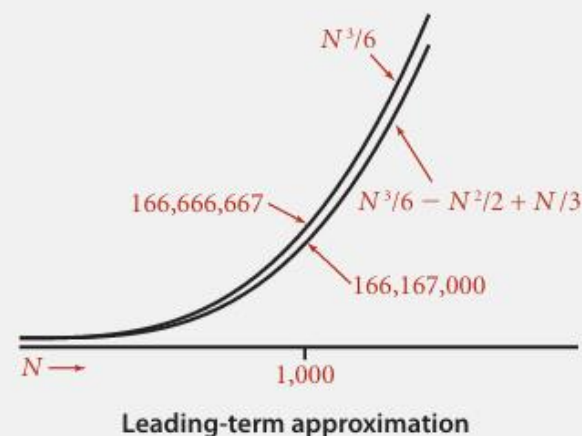
- Estimate running time (or memory) as a function of input size N .
- Ignore lower order terms.
 - when N is large, terms are negligible
 - when N is small, we don't care

Ex 1. $\frac{1}{6} N^3 + 20 N + 16 \sim \frac{1}{6} N^3$

Ex 2. $\frac{1}{6} N^3 + 100 N^{4/3} + 56 \sim \frac{1}{6} N^3$

Ex 3. $\frac{1}{6} N^3 - \underbrace{\frac{1}{2} N^2 + \frac{1}{3} N}_{\text{discard lower-order terms}} \sim \frac{1}{6} N^3$

(e.g., $N = 1000$: 500 thousand vs. 166 million)



Technical definition. $f(N) \sim g(N)$ means $\lim_{N \rightarrow \infty} \frac{f(N)}{g(N)} = 1$

- ♦ **Step 1: Start from an example**
- Suppose the actual running time is:
- $T(n) = 5n^2 + 3n + 2$
- We say “the running time is $O(n^2)$ — ” and we **discard** $3n + 2$.
But why is that mathematically justified?
- ♦ **Step 2: Compare with the dominant term**
 $g(n) = n^2$
- We compute:
- $\lim_{n \rightarrow \infty} \frac{T(n)}{n^2} = \lim_{n \rightarrow \infty} \left(5 + \frac{3}{n} + \frac{2}{n^2}\right) = 5$
- ♦ **Step 3: Interpret the limit**
- Since the limit exists and equals a **positive constant (5)**, we have:
- $T(n) = \Theta(n^2)$
- That means $T(n)$ grows *proportionally* to n^2 — all the smaller-order terms ($3n, 2$) vanish in comparison.
- This is the **mathematical justification** for discarding lower-order terms:
- $\lim_{n \rightarrow \infty} \frac{3n+2}{5n^2} = 0$
- so they become *negligible* as n grows.

♦ **Step 4: Connection to the limit = 1 concept**

If you take only the dominant term $g(n) = 5n^2$, then:

$$\lim_{n \rightarrow \infty} \frac{T(n)}{g(n)} = 1$$

✓ This means $T(n) \sim 5n^2$,
and **asymptotically, both behave identically** — the lower-order terms vanish.

So discarding lower-order terms is equivalent to saying:
“For large n , the ratio between the exact running time and the dominant term approaches 1.”

Simplification 2: tilde notation

- Estimate running time (or memory) as a function of input size N .
- Ignore lower order terms.
 - when N is large, terms are negligible
 - when N is small, we don't care

operation	frequency	tilde notation
variable declaration	$N + 2$	$\sim N$
assignment statement	$N + 2$	$\sim N$
less than compare	$\frac{1}{2} (N + 1) (N + 2)$	$\sim \frac{1}{2} N^2$
equal to compare	$\frac{1}{2} N (N - 1)$	$\sim \frac{1}{2} N^2$
array access	$N (N - 1)$	$\sim N^2$
increment	$\frac{1}{2} N (N - 1)$ to $N (N - 1)$	$\sim \frac{1}{2} N^2$ to $\sim N^2$

Example: 2-SUM

Q. Approximately how many array accesses as a function of input size N ?

```
int count = 0;
for (int i = 0; i < N; i++)
    for (int j = i+1; j < N; j++)
        if (a[i] + a[j] == 0)
            count++;
```

← "inner loop"

A. $\sim N^2$ array accesses.

$$\begin{aligned} 0 + 1 + 2 + \dots + (N-1) &= \frac{1}{2}N(N-1) \\ &= \binom{N}{2} \end{aligned}$$

Bottom line. Use cost model and tilde notation to simplify counts.

Example: 3-SUM

Q. Approximately how many array accesses as a function of input size N ?

```
int count = 0;
for (int i = 0; i < N; i++)
    for (int j = i+1; j < N; j++)
        for (int k = j+1; k < N; k++)
            if (a[i] + a[j] + a[k] == 0)
                count++;
```

"inner loop"

$$\binom{N}{3} = \frac{N(N-1)(N-2)}{3!}$$
$$\sim \frac{1}{6}N^3$$

A. $\sim \frac{1}{2} N^3$ array accesses.

Bottom line. Use cost model and tilde notation to simplify counts.

Estimating a discrete sum

Q. How to estimate a discrete sum?

A1. Take discrete mathematics course.

A2. Replace the sum with an integral, and use calculus!

Ex 1. $1 + 2 + \dots + N$.

$$\sum_{i=1}^N i \sim \int_{x=1}^N x \, dx \sim \frac{1}{2} N^2$$

Ex 2. $1^k + 2^k + \dots + N^k$.

$$\sum_{i=1}^N i^k \sim \int_{x=1}^N x^k \, dx \sim \frac{1}{k+1} N^{k+1}$$

Ex 3. $1 + 1/2 + 1/3 + \dots + 1/N$.

$$\sum_{i=1}^N \frac{1}{i} \sim \int_{x=1}^N \frac{1}{x} \, dx = \ln N$$

Ex 4. 3-sum triple loop.

$$\sum_{i=1}^N \sum_{j=i}^N \sum_{k=j}^N 1 \sim \int_{x=1}^N \int_{y=x}^N \int_{z=y}^N dz \, dy \, dx \sim \frac{1}{6} N^3$$

Mathematical models for running time

In principle, accurate mathematical models are available.

In practice,

- Formulas can be complicated.
- Advanced mathematics might be required.
- Exact models best left for experts.



costs (depend on machine, compiler)

$$T_N = c_1 A + c_2 B + c_3 C + c_4 D + c_5 E$$

A = array access
 B = integer add
 C = integer compare
 D = increment
 E = variable assignment

frequencies
(depend on algorithm, input)

Bottom line. We use **approximate** models in this course: $T(N) \sim c N^3$.

Practice

- How many array accesses does the following code fragment make as a function of n ?
- (Assume the compiler does not optimize away any array accesses in the innermost loop.)
 - `int sum = 0;`
 - `for (int i = 0; i < n; i++)`
 - `for (int j = i+1; j < n; j++)`
 - `for (int k = 1; k < n; k = k*2)`
 - `if (a[i] + a[j] >= a[k]) sum++;`

Solution

- **Step 1: Count (i,j) pairs carefully**
 - We have:
 - for (int i = 0; i < n; i++)
 - for (int j = i+1; j < n; j++)
 - Number of pairs (i,j):
- **Step 2: Innermost loop (k) and comparisons**
 - for (int k = 1; k < n; **k = k*2**)
 - if (a[i] + a[j] >= a[k]) sum++;
 - **k doubles: 1, 2, 4, 8, ... < n $\rightarrow$ number of iterations: $\lfloor \log_2 n \rfloor$**
 - So **total iterations of sum++ per pair** $\approx \log_2 n$.
- **Naively**, total operations:
 - $\frac{n(n-1)}{2} \cdot \log_2 n \approx \frac{1}{2} n^2 \log_2 n$

Solution

- **Step 3:**
- If you **count all array accesses**, not just loop iterations, each iteration of the inner loop may involve **3 array accesses**:
 - $a[i], a[j], a[k]$ in the comparison $a[i] + a[j] \geq a[k]$
- Then, the total **operations $\approx 3 \times (\# \text{ of inner loop iterations})$** :
- $3 \cdot \frac{n(n-1)}{2} \cdot \log_2 n = \frac{3}{2} n^2 \log_2 n$
- **FINAL ANSWER: $\frac{3}{2} n^2 \log n$**

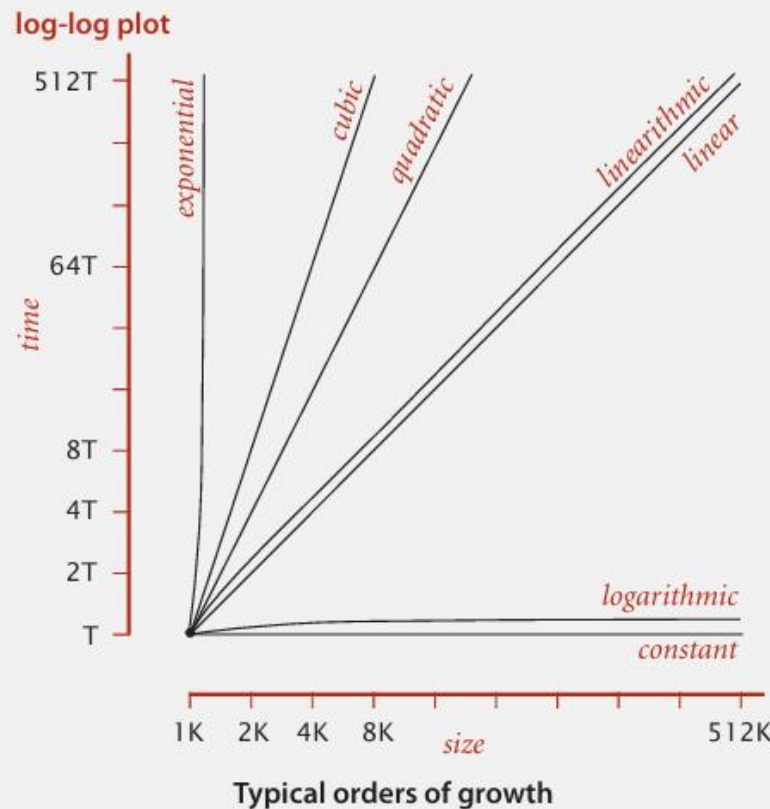
Common order-of-growth classifications

Good news. the small set of functions

1 , $\log N$, N , $N \log N$, N^2 , N^3 , and 2^N

suffices to describe order-of-growth of typical algorithms.

order of growth discards
leading coefficient



Common order-of-growth classifications

order of growth	name	typical code framework	description	example	$T(2N) / T(N)$
1	constant	<code>a = b + c;</code>	statement	add two numbers	1
$\log N$	logarithmic	<code>while (N > 1) { N = N / 2; ... }</code>	divide in half	binary search	~ 1
N	linear	<code>for (int i = 0; i < N; i++) { ... }</code>	loop	find the maximum	2
$N \log N$	linearithmic	[see mergesort lecture]	divide and conquer	mergesort	~ 2
N^2	quadratic	<code>for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) { ... }</code>	double loop	check all pairs	4
N^3	cubic	<code>for (int i = 0; i < N; i++) for (int j = 0; j < N; j++) for (int k = 0; k < N; k++) { ... }</code>	triple loop	check all triples	8
2^N	exponential	[see combinatorial search lecture]	exhaustive search	check all subsets	$T(N)$

Practical implications of order-of-growth

growth rate	problem size solvable in minutes			
	1970s	1980s	1990s	2000s
1	any	any	any	any
$\log N$	any	any	any	any
N	millions	tens of millions	hundreds of millions	billions
$N \log N$	hundreds of thousands	millions	millions	hundreds of millions
N^2	hundreds	thousand	thousands	tens of thousands
N^3	hundred	hundreds	thousand	thousands
2^N	20	20s	20s	30

Bottom line. Need linear or linearithmic alg to keep pace with Moore's law.

Binary search demo

Goal. Given a sorted array and a key, find index of the key in the array?

Binary search. Compare key against middle entry.

- Too small, go left.
- Too big, go right.
- Equal, found.



successful search for 33

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
↑														↑
lo														hi

Live Demo

- <https://yongdanielliang.github.io/animation/web/BinarySearchNew.html>

Reference

- <http://coursera.org/learn/algorithms-part1/>



Thank You

atanu.Shuvam@ulab.edu.bd

University of Liberal Arts Bangladesh
Department of Computer Science & Engineering